

Tecnología Digital VI — Segundo Parcial

Modelo de Examen 1

Universidad Torcuato Di Tella

Duración: 2 horas

Nombre: _____

Parte 1: Verdadero o Falso (2 pts c/u)

Justifique brevemente las respuestas falsas.

- a) Si construimos una red neuronal profunda con múltiples capas ocultas pero sin funciones de activación no lineales, la red resultante es equivalente a una única transformación lineal.
 - b) La función de activación ReLU sufre del problema de *vanishing gradients* porque su derivada es cero para valores negativos.
 - c) En una red convolucional, las capas de Max Pooling tienen parámetros entrenables que se ajustan durante el backpropagation.
 - d) El modelo Word2Vec con arquitectura Skip-Gram recibe una palabra central y predice las palabras de contexto que la rodean.
 - e) En la arquitectura Transformer, el Positional Encoding es necesario porque el mecanismo de Self-Attention procesa todos los tokens en paralelo y pierde la noción de orden secuencial.
 - f) En un modelo Seq2Seq basado en RNNs sin atención, el decoder tiene acceso a todos los hidden states intermedios del encoder para generar la traducción.
-

Parte 2: Multiple Choice (3 pts c/u)

Marque la opción correcta. Solo una es válida.

- 1) ¿Cuál es la principal ventaja de usar **Mini-Batch Gradient Descent** frente a Batch GD y SGD puro?
 - A) Siempre converge al mínimo global de la loss function.
 - B) Combina la estabilidad de Batch GD con la eficiencia computacional de SGD.
 - C) Elimina por completo el problema de los gradientes que se desvanecen.

- D) No requiere definir un learning rate.
- 2) ¿Qué problema resuelven las **skip connections** en ResNet?
- A) Reducen la cantidad de parámetros de la red.
 - B) Permiten que agregar capas nunca empeore el rendimiento, ya que el bloque puede aprender la identidad ($F(x) = 0$).
 - C) Reemplazan la necesidad de funciones de activación no lineales.
 - D) Eliminan la necesidad de usar Batch Normalization.
- 3) En YOLO (v1), ¿cuál es el tamaño del tensor de salida si la imagen se divide en una grilla de $S \times S$, se predicen B bounding boxes por celda y hay C clases?
- A) $S \times S \times (B + C)$
 - B) $S \times S \times (B \times 5 + C)$
 - C) $S \times S \times (B \times 5 \times C)$
 - D) $S \times S \times B \times C$
- 4) ¿Cuál de las siguientes afirmaciones sobre **BERT** y **GPT** es correcta?
- A) BERT usa solo el decoder del Transformer y es autorregresivo.
 - B) GPT usa solo el encoder del Transformer y es bidireccional.
 - C) BERT es bidireccional y se pre-entrena con Masked Language Model; GPT es autorregresivo y predice el siguiente token.
 - D) Ambos modelos usan exactamente la misma arquitectura y difieren solo en los datos de entrenamiento.
- 5) ¿Qué ocurre con Dropout durante la fase de **inferencia**?
- A) Se siguen apagando neuronas al azar igual que en entrenamiento.
 - B) Se desactiva: todas las neuronas participan, y las salidas se escalan.
 - C) Se duplica la tasa de dropout para mayor regularización.
 - D) No tiene efecto porque Dropout solo afecta al optimizador.
-

Parte 3: Desarrollo (10 pts c/u)

1) Cálculo de parámetros CNN vs MLP.

Considere una imagen de entrada de 32×32 píxeles a color (3 canales).

- **Escenario A (MLP):** Se aplica flatten y luego una capa densa de 128 neuronas con ReLU, seguida de una capa de salida de 10 neuronas.
- **Escenario B (CNN):** Se aplica una capa convolucional de 16 filtros de 5×5 (sin padding, stride 1), ReLU, MaxPool 2×2 , y luego una segunda capa convolucional de 32 filtros de 3×3 (sin padding, stride 1).

Para cada escenario:

- a) Calcule el tamaño espacial de la salida después de cada capa usando la fórmula $O = \frac{I-K+2P}{S} + 1$.
- b) Calcule la cantidad total de parámetros entrenables (incluyendo biases).

2) Transfer Learning y Arquitecturas.

Suponga que tiene un dataset de 500 imágenes de rayos X para clasificar entre “sano” y “patológico”. Explique:

- a) ¿Por qué no conviene entrenar una CNN desde cero para este problema?
- b) ¿Cómo aplicaría Transfer Learning usando una red pre-entrenada en ImageNet? Detalle qué capas congelaría y cuáles reemplazaría.
- c) ¿Por qué las primeras capas convolucionales de una red entrenada en ImageNet (fotos naturales) son útiles para imágenes médicas?

3) Attention en Seq2Seq.

Considere una arquitectura Seq2Seq para traducción automática (inglés → español). La frase de entrada tiene 6 palabras.

- a) Explique el problema del “cuello de botella” cuando se usa la arquitectura Encoder-Decoder sin atención.
- b) Describa paso a paso cómo el mecanismo de atención de Bahdanau calcula el vector de contexto c_t al generar la palabra t de la traducción.
- c) ¿Por qué los pesos de atención $\alpha_{t,i}$ deben sumar 1?

Parte 4: Interpretación de Código (15 pts)

Analice el siguiente código de PyTorch y responda las preguntas.

```
1 import torch
2 import torch.nn as nn
3
4 class MiRed(nn.Module):
5     def __init__(self):
6         super().__init__()
7         self.fc1 = nn.Linear(784, 256)
8         self.fc2 = nn.Linear(256, 128)
9         self.fc3 = nn.Linear(128, 10)
10        self.relu = nn.ReLU()
11        self.dropout = nn.Dropout(0.3)
12
13    def forward(self, x):
14        x = x.view(-1, 784)
15        x = self.dropout(self.relu(self.fc1(x)))
16        x = self.dropout(self.relu(self.fc2(x)))
17        x = self.fc3(x)
18        return x
19
20 modelo = MiRed()
21 criterio = nn.CrossEntropyLoss()
```

```

22 optimizador = torch.optim.Adam(modelo.parameters(), lr=0.001)
23
24 for epoch in range(10):
25     modelo.train()
26     for batch_x, batch_y in train_loader:
27         optimizador.zero_grad()
28         salida = modelo(batch_x)
29         loss = criterio(salida, batch_y)
30         loss.backward()
31         optimizador.step()
32
33     modelo.eval()
34     with torch.no_grad():
35         for batch_x, batch_y in val_loader:
36             salida = modelo(batch_x)
37             loss_val = criterio(salida, batch_y)

```

- a) ¿Qué tipo de problema está resolviendo esta red? ¿Para qué dataset parece estar diseñada? Justifique basándose en las dimensiones de entrada y salida.
- b) ¿Qué hace la línea `x = x.view(-1, 784)`? ¿Por qué es necesaria?
- c) ¿Por qué no se aplica Softmax en la capa de salida del `forward`, a pesar de ser un problema de clasificación multiclase?
- d) ¿Por qué se llama a `modelo.train()` antes del entrenamiento y `modelo.eval()` antes de la validación? ¿Qué capa de esta red se comporta distinto entre ambos modos?
- e) ¿Cuál es el propósito de `torch.no_grad()` en el bloque de validación?
- f) ¿Qué hace `optimizador.zero_grad()` y qué pasaría si se omitiera?